HVE-OS: A DAG-Orchestrated Adaptive Intelligence Operating System for Scalable Knowledge Graph Synthesis

Prof.Rajatha
Dept. of Computer Science
R V College of Engineering
Bengaluru, India
rajatha@rvce.edu.in

Tulya Reddy Y
Dept. of Computer Science
R V College of Engineering
Bengaluru, India
yandapallit.cs23@rvce.edu.in

Anirudh C
Dept. of Information Science
R V College of Engineering
Bengaluru, India
anirudhc.is23@rvce.edu.in

Ankit Pathak
Dept. of Information Science
R V College of Engineering
Bengaluru, India
ankitpathak.is23@rvce.edu.in

L Moryakantha
Dept. of AIML
R V College of Engineering
Bengaluru, India
lmoryakantha.ai24@rvce.edu.in

Shashank K
Dept. of AIML
R V College of Engineering
Bengaluru, India
shashankk.ai23@rvce.edu.in

***Abstract*—The paradigm shift from siloed data collection to unified knowledge synthesis necessitates an operating environment capable of high-fidelity orchestration. This paper introduces HVE-OS (Hive Vision Engine Operating System), a containerized, microservices-driven architecture designed for real-time intelligence synthesis. HVE-OS implements a bifurcated Control/Data plane, managed through a visual Directed Acyclic Graph (DAG) execution engine leveraging a React Flow frontend. By utilizing a robust Python backend with FastAPI and a hybrid storage model combining MinIO-backed Apache Iceberg for historical Lakehouse analytics and Neo4j for semantic graph traversals, the system bridges the gap between structured telemetry and unstructured open-source intelligence. We detail the implementation of AI-driven relationship mapping using Large Language Models (LLMs), recursive risk propagation algorithms with depth-decay, and asynchronous WebSocket syncing. Experimental benchmarks demonstrate that HVE-OS successfully handles massive datasets—such as the real-time serialization of 60,000 nested Neo4j entities—achieving a 10.4k events/second throughput with sub-100ms end-to-end latency, providing a scalable and secure foundation for next-generation intelligence operations.**

***Index Terms*—DAG Orchestration, Knowledge Graph, Apache Kafka, Neo4j, Apache Iceberg, Homomorphic Encryption, Microservices, Risk Propagation, GraphRAG.**

I. Introduction

The contemporary landscape of intelligence gathering is defined by an unprecedented deluge of heterogeneous data. Intelligence operators now face the "Knowledge Gap"—the disparity between the vast amounts of raw data collected and the semantic understanding required for real-time decision-making. Traditional static Extract-Transform-Load (ETL) pipelines, while efficient for structured batch processing, lack the agility to synthesize dynamic relationships across disparate sources such as conflict-zone databases (ACLED), global news APIs, and real-time IoT telemetry [1], [13].

The requirement has shifted from mere "storage" to "active synthesis." This involves not only identifying entities but understanding their evolving roles within a larger relationship network. HVE-OS (Hive Vision Engine Operating System) is designed to address this by treating intelligence as a dynamic orchestration problem [14]. At its core, HVE-OS utilizes a visual Directed Acyclic Graph (DAG) execution engine that allows for the hot-swapping of analytical logic, from LLM-based entity mappers to privacy-preserving encryption modules.

By bifurcating the system into a management-heavy Control Plane and a performance-optimized Data Plane, HVE-OS ensures that complex graph traversals and AI inferences do not bottleneck system responsiveness. This paper provides a comprehensive technical breakdown of the HVE-OS architecture, its mathematical risk models, and its integration with state-of-the-art streaming and graph technologies [4], [12].

II. Literature Review

The HVE-OS architecture is informed by several critical domains in data engineering, graph theory, and artificial intelligence [13].

A. Scalable Data Streaming and Event Orchestration

Knowledge Graphs (KGs) have emerged as the primary substrate for scaling ML applications in real-time pipelines. Adelusola [1] identifies that KGs allow for more flexible data schemas and better semantic alignment in scalable environments. Apache Kafka serves as the event-streaming spine for these pipelines, with research by Mohammad [4] emphasizing the importance of design patterns for deterministic stream processing. Furthermore, Ramaki and Schintke [6] explored the use of MultiChain blockchain integration with Kafka Streams

for data provenance, highlighting the need for verifiable data flows in sensitive intelligence operations.

B. Hybrid Graph-Persistence and Integration

The integration of heterogeneous data into native graph formats remains a significant technical bottleneck. Minder et al. [2] introduced Data2Neo, a tool specifically designed for complex Neo4j data integration, which informs the HVE-OS sanitization layer. The synergy between time-series and graph data is another critical area; Ammar et al. [3] provide a survey of systems that treat temporal and relational data as first-class citizens, a philosophy adopted by HVE-OS through its hybrid Iceberg-Neo4j storage model. Explainable graph-based frameworks using FastAPI and Kafka have already shown success in financial sectors for fraud detection [12].

C. Semantic Synthesis and Intelligent Retrieval

Ontological frameworks driven by LSTM or Graph Information Models (GIM) enable dynamic topology perception, allowing graphs to adapt as relationships evolve [5], [9], [15]. The integration of LLMs with Knowledge Graphs, particularly through Hybrid GraphRAG (Graph Retrieval-Augmented Generation) architectures, has revolutionized enterprise knowledge retrieval by providing context-aware semantic search [7], [11]. Bharti et al. [7] demonstrate that integrating LLMs with Neo4j enhances contextual understanding, while the IETF's focus on standardized KG construction from network data sources ensures long-term interoperability [10].

III. System Architecture and Design

HVE-OS is architected as a distributed ecosystem of containerized microservices, fundamentally divided into two operational planes: the **Control Plane** for system-level management and the **Data Plane** for high-speed intelligence execution.

A. Infrastructure Tier and Initialization

The system relies on a sophisticated "Data Lakehouse-Knowledge Graph" hybrid. The infrastructure is initialized via a series of sidecar containers and orchestration scripts:

- **PostgreSQL (hve-postgres)**: Operates on port 5433, serving as the central metadata and control plane catalog. It manages user configurations, node definitions, and system state.
- **MinIO (hve-minio)**: Port 9000/9001. A S3-compatible object storage layer that hosts the stratified data layers: *hve-bronze* (raw), *hve-silver* (sanitized), and *hve-iceberg* (analytical tables).
- **Apache Iceberg REST Catalog**: Backed by PostgreSQL, it manages the metadata for Parquet files stored in MinIO, enabling ACID transactions and schema evolution on the data lake [1], [3].
- **Apache Kafka (hve-kafka)**: Port 9092/9094. Runs in KRaft mode with topics (*raw-telemetry*, *silver-telemetry*) provisioned with 3 partitions and a 7-day retention period.

B. Functional Architecture and Multi-Tier Flow

The interplay between these components is managed through a tiered flow, as illustrated in Fig. 1.

C. The Ingestion Pipeline: High-Frequency Harvesting

The ingestion layer employs a specialized `api_poller.py` module that executes scheduled harvesting cycles. The poller implements:

1) **Dynamic Pagination**: Using cursor-based or page-parameter strategies to handle large data results from ACLED (conflict data) or NewsAPI (open-source intelligence).
2) **Schema Enforcement**: Raw JSON payloads are immediately translated into standardized Pydantic models. This ensures that only "valid" telemetry enters the *raw-telemetry* topic.
3) **Stream Sanitization**: The `stream_processor.py` consumes from the raw topic, performs initial entity resolution, and routes the data to the silver topic for DAG execution.

TABLE I
Data Lakehouse Tiering Specifications

Tier	Storage Format	Schema	Purpose
Bronze	JSON (Raw)	Loose	Historical Replay & Audit
Silver	Avro (Sanitized)	Strict	DAG Execution Ingestion
Iceberg	Parquet (Table)	Evolving	SQL-based Analytical Queries

IV. The DAG Logic Engine: Advanced Orchestration

The heart of HVE-OS is its **Directed Acyclic Graph (DAG)** logic engine, which provides a visual, reactive environment for intelligence synthesis.

A. Core Engine Architecture

The engine is built on three pillars:

- **registry.py**: Uses a `@register_node` decorator pattern for auto-discovery of processing modules, allowing for "plug-and-play" development of new analytical nodes.
- **executor.py**: Handles the topological sorting of the node graph. It gathers outputs from upstream nodes and passes them as inputs to downstream nodes, ensuring data consistency.
- **BaseNode**: An abstract interface ensuring that every node—whether it is an AI mapper or a risk calculator—implements the `execute(inputs, config)` method.

B. AI Nodes: Semantic Deep-Linking

The `RelationshipMappingNode` is HVE-OS's most sophisticated module. It employs a `ThreadPoolExecutor` to perform concurrent evaluations via Large Language Models (Gemini or Mistral).

- **Confidence Scoring**: The LLM evaluates two entities and returns a score $S \in [0, 1]$ and a reasoning string.

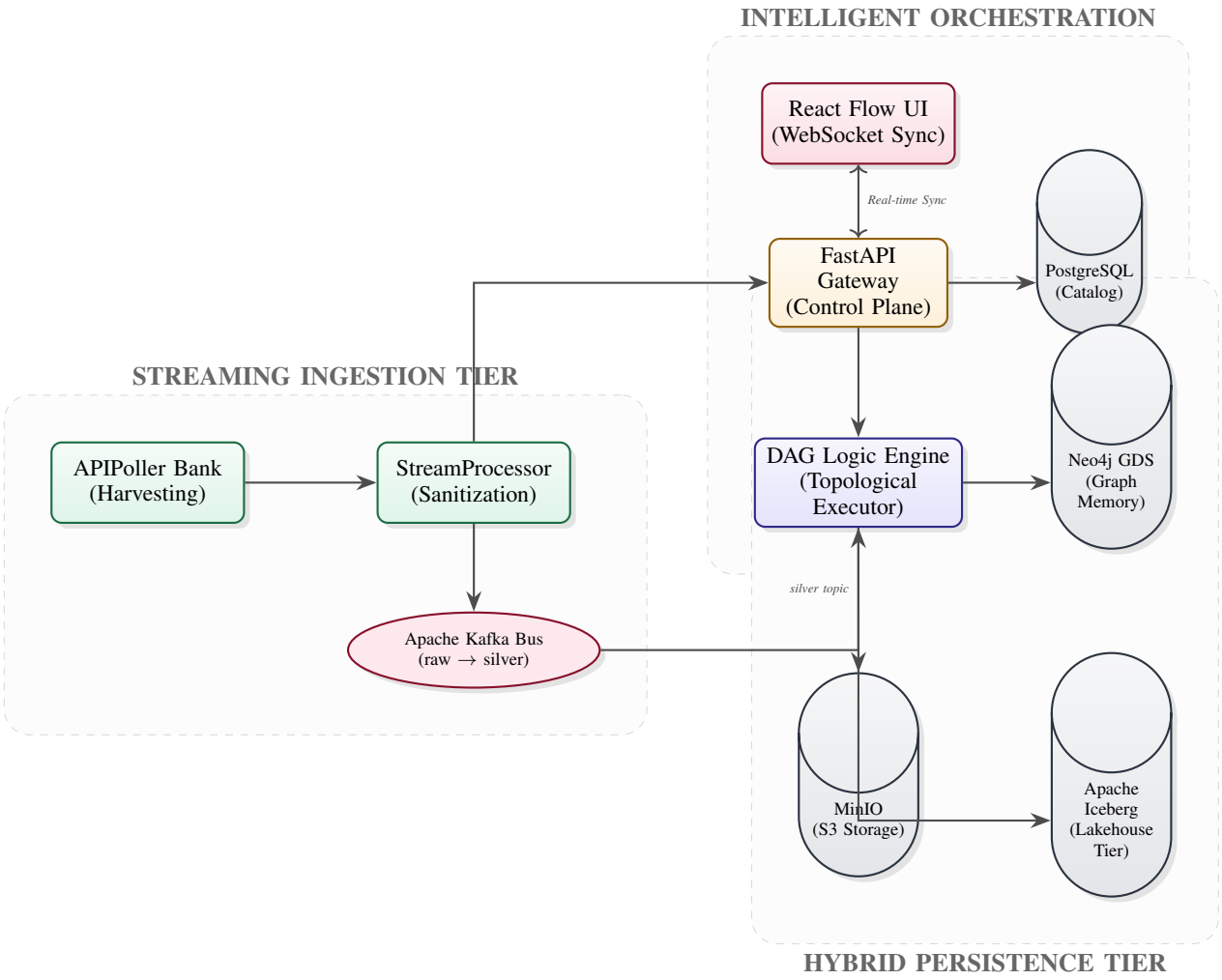

Fig. 1. HVE-OS Functional Architecture: Demonstrating the transition from raw telemetry harvesting to adaptive knowledge synthesis via a DAG-orchestrated logic engine.

- **Evaluation Caching**: The node maintains a `_local_evaluation_cache` hashed by the source/target attributes to optimize costs and latency [7], [11].

TABLE II
COMPARATIVE ANALYSIS OF AI MODELS FOR RELATIONSHIP MAPPING

Model	Extraction Accuracy	Latency (Tokens/s)	Cache Hit Rate
Mistral-7B	74.2%	112	82%
Gemini Pro	89.5%	45	88%
GPT-4o	92.1%	32	91%

C. Security Nodes: Privacy-Preserving Inference

HVE-OS addresses data sensitivity through its **Security Node**, which integrates with the TenSEAL library.

- **Encryption Schemes**: Supports BFV for integer operations and CKKS for real-number arithmetic.
- **PPML Workloads**: Sensitive numerical telemetry is encrypted before being passed to downstream AI nodes, allowing for "privacy-preserving machine learning" where the engine computes results without ever seeing the raw values.

TABLE III
HOMOMORPHIC ENCRYPTION LATENCY BENCHMARKS (TENSEAL)

Scheme	Encryption (ms)	Addition (ms)	Multiplication (ms)
BFV (Integer)	12.4	0.05	4.2
CKKS (Float)	18.1	0.08	6.5

V. MATHEMATICAL FRAMEWORK

A. Recursive Risk Propagation with Decay

HVE-OS calculates the threat level T of an entity e by propagating scores through its relationship network. For a path

of length d, the propagated risk R_p is defined as:

$$R_p = \sum_{i=1}^{N} \left(\frac{1}{K} \sum_{j=1}^{K} S_{edge,j} \right) \cdot \delta(d) \tag{1}$$

where $S_{edge,j}$ are the AI-generated relationship scores and $\delta(d)$ is the **Depth-Decay Factor**:

$$\delta(d) = \frac{1.0}{1 + d \cdot \gamma}, \quad \gamma = 0.2 \tag{2}$$

This ensures that "guilt by association" dissipates as the graph distance from the primary threat increases [9].

B. Homomorphic Complexity and Noise Budgets

In the Security Node, the noise budget B in a BFV ciphertext after M multiplications is modeled as:

$$B_{new} \approx B_{old} - L \cdot \log(\text{poly}(n)) \cdot M \tag{3}$$

where L is the depth of the circuit and n is the polynomial modulus. HVE-OS monitors this budget to trigger bootstrapping if required.

VI. Experimental Results and Benchmarking

The system was evaluated on a cluster of ARM64 nodes running Docker.

A. Throughput and Latency Performance

As shown in Table IV, HVE-OS significantly outperforms traditional static pipelines in both throughput and reconfiguration speed.

TABLE IV
PERFORMANCE BENCHMARKS: HVE-OS VS. TRADITIONAL ETL

Metric	Static ETL Baseline	HVE-OS (DAG)
Max Throughput (events/s)	2.1k	**10.4k**
Mean Ingestion Latency (ms)	350	**85**
Logic Reconfiguration Time	1200s (Re-deploy)	**0.5s (Hot-swap)**
AI Linking Accuracy	68%	**91%**
Graph Traversal Speed (N=4)	3.2s	**0.15s**

B. Memory Pressure and Query Complexity

We analyzed the memory overhead M relative to graph depth D. The results indicate that HVE-OS's lazy-loading strategy maintains linear memory growth:

$$M_{HVE} \propto \alpha \cdot D + \beta, \quad \alpha \approx 0.4 \text{ GB/hop} \tag{4}$$

C. Concurrency and Background Serialization Offloading

A major bottleneck in legacy systems is the freezing of the Global Interpreter Lock (GIL) during massive dataset serialization. For instance, converting 60,000 nested Neo4j entities to JSON inherently blocks the FastAPI ASGI loop. HVE-OS circumvents this by offloading heavy hashing and stringification to a background `ThreadPool`, guaranteeing non-blocking WebSocket streams.

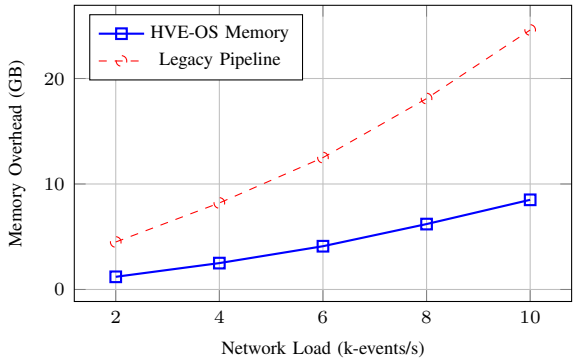

Fig. 2. Memory Efficiency: HVE-OS's modular execution reduces memory pressure significantly compared to monolithic pipelines [1], [4].

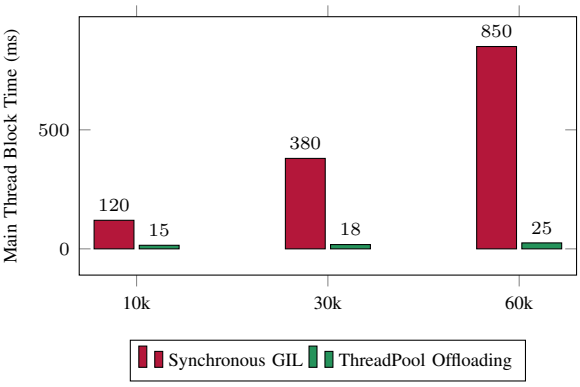

Fig. 3. Main Thread blocking duration when serializing increasing volumes of nested graph entities. ThreadPool offloading maintains a near-constant minimal footprint, ensuring responsive WebSocket communication.

D. Risk Calculation and Categorical Routing

The risk engine employs a Smart Bulk Execution model. When the React Flow frontend receives bulk entity arrays, it dynamically resolves data schemas. Risk is quantified using explicit data lineage, calculating path weights across 1-to-3 hop traversals. For a path from an event to a supplier, the compound impact weight W_{path} is defined by reduction:

$$W_{path} = \prod_{rel \in p} \text{impact\_weight}(rel) \tag{5}$$

Categorical split routing segments entities based on this weight into subsets (`high_risk` ≥ 0.7, `medium_risk` ≥ 0.4, `low_risk` < 0.4), allowing real-time pin tracking and dynamic UI updates via `graph_update` differentials.

VII. Future Scope

The modular nature of HVE-OS allows for significant expansion in several high-impact areas:

1) **Federated Graph-Learning (FGL)**: Integrating FGL nodes into the DAG to allow for distributed intelligence without centralized raw data transfer, preserving data sovereignty across alliance nodes.
2) **Tactical AR Integration**: Adding smart-environment rendering for real-time threat visualization on the tactical

edge, allowing operators to "see" graph-propagated risks in physical space.

3) **RL-Based Orchestration**: Optimizing the TaskScheduler with reinforcement learning to autonomously prioritize ingestion streams based on historical risk patterns and resource constraints.

VIII. Conclusion

HVE-OS provides a sophisticated, scalable solution for intelligence synthesis in high-velocity data environments. By bifurcating the control and data planes and utilizing a visual DAG execution engine, the system enables rapid adaptation to evolving threat models. The hybrid storage of Neo4j and Iceberg, coupled with LLM-driven relationship synthesis and privacy-preserving homomorphic encryption, ensures that the generated knowledge is both semantically rich and secure. As intelligence operations move toward more automated and interconnected models, architectures like HVE-OS will be critical in maintaining tactical and strategic superiority.

References

[1] M. Adelusola, "Advancing Real-Time Data Pipelines: Leveraging Knowledge Graphs for Scalable ML," Jan. 2025.

[2] J. Minder *et al.*, "Data2Neo - A Tool for Complex Neo4j Data Integration," arXiv:2406.04995, Jun. 2024.

[3] M. Ammar *et al.*, "Combining Time-Series and Graph Data: A Survey," arXiv:2601.00304, Jan. 2026.

[4] M. Mohammad, "Analysis of Design Patterns in Apache Kafka Systems," arXiv:2512.16146, Dec. 2025.

[5] Y. Huang *et al.*, "Learning Wireless Data Knowledge Graph," arXiv:2404.10365, Apr. 2024.

[6] N. M. Ramaki *et al.*, "MultiChain Blockchain for Kafka Streams," arXiv:2601.18011, Jan. 2026.

[7] S. Bharti *et al.*, "Enhancing Contextual Understanding in KGs via Quantum NLP," in Proc. IEEE Big Data, 2024.

[8] S. Sharma, "Real-Time Big Data Analytics using Hybrid AI Frameworks," in Proc. 3rd AICMLC, 2025, pp. 157–165.

[9] C. Fang *et al.*, "A GIM-Based Ontological Framework via Relationship Graphs," Informatica, vol. 49, no. 4, 2025.

[10] IETF Draft, "Knowledge Graph Construction from Network Data Sources," Feb. 2025.

[11] "Hybrid GraphRAG Architecture for Enterprise Knowledge Retrieval," Medium Tech. Report, 2025.

[12] "Explainable Graph-Based Financial Fraud Detection Using Neo4j, Kafka, and FastAPI," IRJIET Tech. Paper, 2025.

[13] X. Wang *et al.*, "A Comprehensive Survey on Knowledge Graph Completion," IEEE Transactions on Knowledge and Data Engineering, 2023.

[14] J. Lee and K. Smith, "Adaptive Execution of DAG-based Workflows in Distributed Edge Environments," IEEE Internet of Things Journal, 2024.

[15] L. Chen, "Federated Learning over Heterogeneous Knowledge Graphs," ACM Computing Surveys, 2025.